Frontend Architecture & Optimization Guide

Gema Event Management Platform
Generated: 2026-02-25

1. Overview

The GEMA frontend is built with React 18, Vite, and TypeScript. It relies heavily on modern React features like Context API, automatic batching, and `useMemo`/`useCallback` for performance.

2. Performance Engineering

The frontend underwent a major 3-phase optimization journey resulting in significant Core Web Vital improvements:

- **Bundle Size:** ~690KB (-13.8% reduction)
- **FCP (First Contentful Paint):** 1.5-2s
- **TBT (Total Blocking Time):** < 300ms
- **CLS (Cumulative Layout Shift):** < 0.05
- **Route Navigation:** ~150ms

2.1 Context Management

- **Rule:** Combine related contexts to avoid render cascades.
- **Implementation:** `PreferencesContext` manages Theme, Language, and Currency. This avoids 7 levels of context nesting, reducing initial render times by ~40%.

2.2 Animation Strategies

- **Rule:** Prefer CSS animations over JavaScript animation libraries (`react-spring` was removed).
- **Implementation:** Custom `animations.css` handles fades, slides, and scaling. Framer Motion is lazy-loaded only when strictly necessary for complex interactions.

2.3 List Memoization

- **Rule:** Always memoize list items (e.g., `EventCard`).
- **Implementation:** Uses `React.memo` with custom comparison functions. Props passed to cards are memoized with `useMemo` and callbacks with `useCallback`.

2.4 Suspense and Skeletons

- **Rule:** Use layout-aware skeleton loaders instead of generic spinners.
- **Implementation:** Implemented route-specific skeleton components (`HomePageSkeleton`, `CheckoutPageSkeleton`, etc.) to practically eliminate CLS.

2.5 Debouncing

- **Rule:** Debounce expensive user inputs (especially Search) to prevent main-thread blocking.
- **Implementation:** 300ms sweet spot for search inputs combined with `useTransition` for non-blocking UI updates.

3. UI Implementation Patterns

3.1 Conditional Forms (Event Creation)

Forms heavily rely on reactive conditional fields based on event types:

- **Offline Events:** Requires Address/Coordinates.
- **Online/Hybrid Events:** Requires Meeting Links.
- **Location Meta (SEO):** Country and City are explicitly stored for logical mapping.

3.2 Routing Structure

- Utilizing React Router with consolidated routes. (e.g. `booking/:eventId` handles what used to be 4 disparate routes).
- Avoid `AnimatePresence` on route transitions as it blocked rendering for 100-200ms.

4. Development Standards

- Maintain Strict Mode compliance.
- No `console.errors` or unresolved TypeScript issues.
- Consolidate layout logic into explicit components instead of inline styling permutations.